

UNIT-1

What is an Algorithm?

An algorithm is a finite sequence of instructions for solving a computational problem.

In addition, all algorithms must satisfy the following criteria:

1. **Input:** Zero or more quantities are externally supplied.
2. **Output:** At least one quantity is produced as output.
3. **Definiteness:** Each instruction must be clear and unambiguous.
4. **Finiteness:** The algorithm must terminate after a finite number of steps.
5. **Effectiveness:** Each instruction must be feasible, that means a person should be able to carry out the instruction correctly by hand in a finite length of time. And algorithm must not contain unnecessary or redundant instructions.

In formal computer science, one distinguishes between an algorithm and a program.

A program does not necessarily satisfy the 4th condition (finiteness). One important example for such a program for a computer is its OS, which never terminates except for system crashes or when system is turned off, but continues to loop until a new job is entered.

Since our programs always terminate, we use algorithm and program interchangeably.

Pseudocode conventions for expressing Algorithms:

1. Comments begin with `//` and continue until the end of line.
2. Blocks are indicated with matching braces: `{` and `}`.
 - Statements are delimited by `;`.
3. An identifier begins with a letter.
 - The data types of variables are not explicitly declared. The types will be clear from the context. Whether a variable is global or local to a procedure will also be evident from the context.
4. Compound data types can be formed with records.

Example:

```
node = record
{
datatype_1 data1;
datatype_2 data2;
:
datatype_n data-n;
node *link;
}
```

- Here, link is a pointer to the record type node.

5. Assignment of values to variables is done using the assignment statement.
`<variable>: = <expression>;`

6. There are two Boolean values **true** and **false**.
Logical operators: *AND*, *OR*, and *NOT*
Relational operators: `<`, `>`, `≤`, `≥`, `=`, `≠`

7. Elements of multidimensional arrays are accessed using `[` and `]`.

Example:

If A is a 2D array, the (i , j)th element of the array is denoted as A[i , j].

- Array indices start at 1.

8. The following looping statements are employed:
for, **while**, and **repeat-until**.

- The **while** loop takes the following form:

```
while<condition> do
{
<statement 1>
...
<statement n>
}
```

- The general form of **for** loop:

```

for variable := value1 to value2 step step_size do
{
<statement 1>
...
<statement n>
}

```

The clause “**step** *step_size* ” is optional and taken as +1 if it is not present. *step_size* could either be positive or negative.

9. A **repeat-until** statement is constructed as follows:

```

repeat
<statement 1>
...
<statement n>
until<condition>

```

The statements are executed as long as<*condition*> is false. The value of *condition* is computed after executing the statements.

- The instruction **break**; can be used within any of the above looping instructions to force **exit**.

10. A conditional statement has the following forms:

- **if**<*condition*> **then** <*statement*>
- **if**<*condition*> **then** <*statement 1*> **else** <*statement 2*>
- **case statement**

```

case
{
: <condition 1> : <statement 1>
: <condition 2> : <statement 2>
...
: <condition n> : <statement n>
: else: <statement n+1>
}

```

11. Input & output are done using the instructions **read** and **write**.

12. There is only one type of procedure (or, function): **Algorithm**

- An algorithm consists of a heading and a body.
- The heading takes the form:
Algorithm Name (<parameter list >)
- The body has one or more statements enclosed within braces.
- An algorithm may or may not return any values.
- Simple variables to procedures are passed by values.
- Arrays and records are passed by reference.
An array name or a record name is treated as a pointer to the respective datatype.

Example: Write an algorithm that finds and returns the maximum of ‘n’ given numbers.

Solution:

```

Algorithm Max(A, n)
// A is an array of size n.
{
    Result := A[1];
    for i := 2 to n do
        if A[i] > Result then Result := A[i];
    return Result;
}

```

In the above algorithm, ‘A’ and ‘n’ are procedure parameters. ‘Result’ and ‘i’ are local variables.

SELECTION SORT

Suppose we want to devise an algorithm that sorts a collection of n elements of arbitrary type.

A Simple solution is given by the following statement:

“From those elements that are currently unsorted, find the smallest and place it next in the sorted list.”

The above statement is not an algorithm because it leaves several questions unanswered. For example, it doesn't tell us “where and how the elements are initially stored, or where we should place the result.”

We assume that, the elements are stored in an array ‘a’ such that the i^{th} element is stored in the i^{th} position, i.e., $a[i]$, $1 \leq i \leq n$.

And we also assume that the sorted elements are also stored in the same array ‘a’.

Algorithm:

```
for i := 1 to n do
{
    Examine  $a[i]$  to  $a[n]$  and suppose the smallest element is at  $a[j]$ ;
    Interchange  $a[i]$  and  $a[j]$ ;
}
```

To turn the above algorithm into a pseudocode program, two clearly defined subtasks remain:

1. Finding the smallest element (say, $a[j]$).
2. Interchanging it with $a[i]$.

Note: To denote the range of array elements, $a[1]$ through $a[n]$, we use the notation $a[1:n]$.

Algorithm SelectionSort (a, n)

```
{
    // sort the array  $a[1:n]$  into non decreasing order.
    for i := 1 to n-1 do
    {
        //Find out the index of the smallest element from  $a[i:n]$  and place it in  $j$ .
        j:= i;
        for k := i+1 to n do
        {
            if ( $a[k] < a[j]$ ) then
            { j:= k; }
        }
        if ( $j \neq i$ ) then
        {
            // interchange  $a[i]$  and  $a[j]$ 
            t :=  $a[i]$ ;
             $a[i]$  :=  $a[j]$ ;
             $a[j]$  := t;
        }
    }
}
```

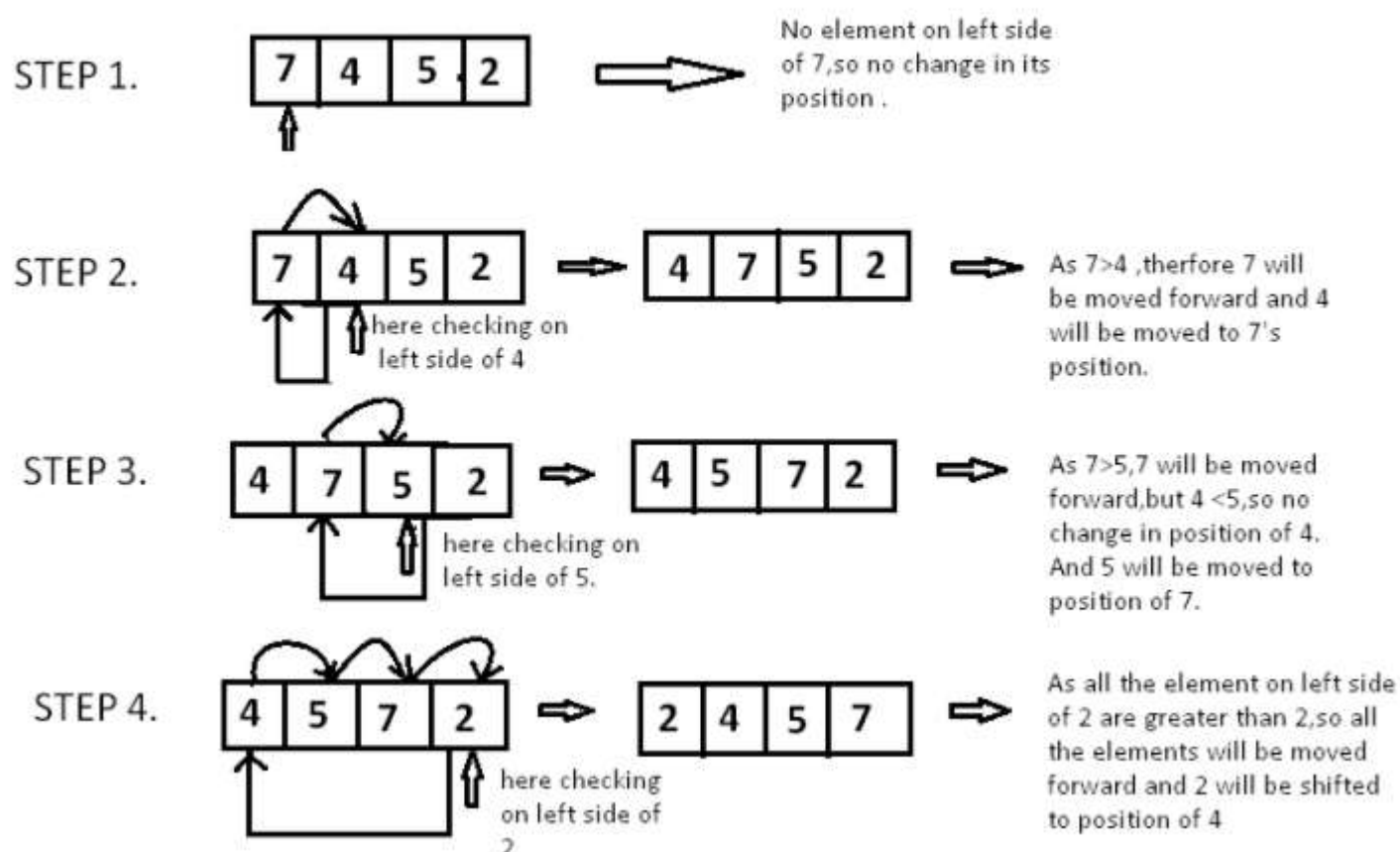
Algorithm for Insertion Sort:

Algorithm InsertionSort (a, n)

```
{
    // sort the array a[1:n] into non decreasing order.
    for i := 2 to n do
    {
        key=a[i];
        // Insert a[i] into the sorted part of the array, i.e., a [1: i-1]
        j=i-1;
        while(j>=1 and a[j]>key) do
        {
            a[j+1]=a[j]; // move a[j] to its next position in the right side
            j=j-1;
        }
        a[j+1]=key;
    }
}
```

Example:

Take array $A[] = [7, 4, 5, 2]$.



Performance analysis:-

To judge the performance of an algorithm we use two terms.

1) Space complexity

2) Time complexity

1) Space complexity:-

The space complexity of an algorithm is the amount of memory it needs to run to completion.

2) Time complexity:-

The time complexity of an algorithm is the amount of computer time it needs to run to completion.

Space complexity:-

The space needed by an algorithm is seen to be the sum of the following components.

→ A fixed part that is independent of the characteristics of the inputs and output. This part typically includes the instruction space, space for simple variables & fixed size component variables, space for constants and so on.

→ Variable part that consists of the space needed by component variables whose size is dependent on the particular problem instance being solved, the space needed by reference variables (to the extent that this depends on instance characteristics), and the recursion stack space.

→ The space requirement $S(P)$ of any algorithm P may therefore be written as,

$$S(P) = C + S_p(\text{instance characteristics}).$$

Where, C is a constant.

→ When analysing the space complexity of an algorithm, we concentrate solely on estimating $S_p(\text{instance characteristics})$.

For any given problem, first we need to determine which instance characteristics to use to measure the space requirements. Generally speaking, our choices are limited to quantities related to the number and size of the input and outputs of the algorithm.

EXAMPLE-1:-

```
Algorithm abc(a, b, c)
{
    return a+b+b*c+(a+b-c)/(a+b)+40;
}
```

→ For this algorithm, the problem instance is characterised by the specific values of a, b, c .

Assume that one word is sufficient to store the values of each a, b, c and the result. So, the space needed by algorithm abc is 4 words. So, the space needed by abc is independent of the instance characteristics. Consequently, $S_p(\text{instance characteristics})$ is equal to 0.

So, space needed by this algorithm is constant.

EXAMPLE 2: Iterative algorithm for sum of n real numbers

```
Algorithm sum (a, n)
{
    S=0.0;
    for i:=1 to n do
        S:=S+a[i];
    return S;
}
```

For this algorithm, the problem instance is characterised by n (means value of n).

The space needed for array address ' a ' is one word. The space needed by n is one word. Space needed by i and S are one word and one word.

So, we obtain $S_{\text{sum}}(n) = 0$.

So, space complexity of the algorithm sum is, $S(\text{sum}) = \text{constant}$.

EXAMPLE-3:-

Recursive algorithm for sum of n numbers:

```
Algorithm Rsum(a, n)
{
    if (n<=0) then return 0.0;
    else return Rsum(a, n-1) + a(n);
}
```

For this algorithm, the problem instance is characterised by n .

The recursion stack space includes space for the formal parameters, local variables, and the return address. Assume that return address requires one word of memory and one word is required for each of the formal parameters ' a ' and ' n '. Since the depth of the recursion is n , the recursion stack space needed is $3n$.

So, $S_{\text{Rsum}}(n) = 3n$.

$S(\text{Rsum}) = C + 3n = O(n)$.

EXAMPLE-4:-

Algorithm copy(a, n)

```
{
    for i:=1 to n do
        b[i]:=a[i];
}
```

Array ' b ' needs ' n ' memory locations.

So, $S_{\text{copy}}(\text{instance characteristics}) = S_{\text{copy}}(n) = n$.

$S(\text{copy}) = C + n = O(n)$.

Time complexity:-

The time $T(P)$ taken by an algorithm P , is the sum of the compile time and run time.

→ Compile time is fixed. It does not depend on the instance characteristics.

→ Consequently, we concern ourselves with just the run time of an algorithm.

→ The runtime of algorithm P is denoted by $t_p(\text{instance characteristics})$, where instance characteristics are those parameters that characterize the problem instance .

Determining the time complexity by step count method:

→ The time complexity of an algorithm is given by the number of steps taken by the algorithm to compute the function for which it was written.

→ The no. of steps is itself a function of the instance characteristics.

Usually, we choose those characteristics that are of importance for us.

→ Once the relevant instance characteristics (such as $n, m, p, q, \dots$) have been selected, we can define what a step is.

→A step is any computational unit that is independent of the instance characteristics (n, m, p, q, ...).

EXAMPLE:-

- ❖ 10 additions can be one step.
- ❖ 100 multiplications is one step.

But

- ❖ n additions cannot be one step, or
- ❖ m/2 additions cannot be one step, or
- ❖ p+q subtractions cannot be one step.

Method to determine the step count of an algorithm:-

- (1) Determine the total no. of times each statement is executed (i.e., frequency).
- (2) Determine the no. of steps per execution(s/e) of the statement.
- (3) Multiply the above two quantities to obtain the total no. of steps contributed by each statement.
- (4) Add the contributions of all statements to obtain the step count for the entire algorithm.

EXAMPLE-1:-

Statement	No. of times of execution (frequency)	s/e	total no. of steps per statement
Algorithm sum(a,n) { s:=0.0; for i:=1 to n do s:=s+a[i]; return s; }	1 n+1 n 1	1 1 1 1	1 n+1 n 1
Total step count of algorithm =			2n+3

So, $t_{sum}(n)=2n+3$.

The step count tells us how the runtime for a program changes in the instance characteristics.

→From the step count for the above algorithm sum, we see that, if n is doubled, the runtime also doubles(approximately).

EXAMPLE-2:-RSum algorithm:-

Assume the time complexity of Rsum is $t_{Rsum}(n)$.

Statement	No. of times of execution (frequency)		s/e	total no. of steps per statement	
	n<=0	n>0		n<=0	n>0
Algorithm Rsum(a,n) { if (n<=0) then return 0.0; else return Rsum (a,n-1)+ a(n); }	1 1 0	1 0 1	1 1 $1+t_{Rsum}(n-1)$	1 1 0	1 0 $1 + t_{Rsum}(n-1)$
Total Step Count =				2	$2+ t_{Rsum}(n-1)$

When analysing a recursive algorithm for its step count, we often obtain a recursive formula for the step count. For the above algorithm,

$$t_{Rsum}(n) = \begin{cases} 2, & \text{if } n \leq 0 \\ 2 + t_{Rsum}(n - 1), & \text{if } n > 0 \end{cases}$$

→These recursive formulas are referred to as recurrence relations.

→One way of solving any such recurrence relation is to make repeated substitutions for each occurrence of the function or term t_{Rsum} on the RHS until all such occurrences disappear.

Solving the above recurrence relation:

$$\begin{aligned} t_{Rsum}(n) &= 2+t_{Rsum}(n-1) \\ t(n) &= 2+t(n-1) \\ &= 2+(2+t(n-2)) \end{aligned}$$

=2+2+(2+t(n-3))

=2*3+t(n-3)

After k substations,

t(n)=2*k + t (n-k)

If n=k, then

t(n) = 2*n + t(0)

= 2n + 2

t_{Rsum} (n)=2n+2

EXAMPLE-3:- Addition of two mxn matrices

Statement	No. of times of execution (frequency)	s/e	total no. of steps per statement
Algorithm Add(a,b,c,m,n) { for i:=1 to m do for j:=1 to n do c[i,j]:=a[i,j]+ b[i,j]; } 	m+1 m(n+1) mn	1 1 1	m+1 mn+ m mn
Total step count =			2mn+2m+1

EXAMPLE-3a:- Addition of two nxn matrices

Statement	No. of times of execution (frequency)	s/e	total no. of steps per statement
Algorithm Add(a,b,c,n) { for i:=1 to n do for j:=1 to n do c[i,j]:=a[i,j]+ b[i,j]; } 	n+1 n(n+1) n²	1 1 1	n+1 n²+ n n²
Total step count =			2n ² +2n+1

EXAMPLE-3b:- Multiplication of two nxn matrices

Statement	No. of times of execution (frequency)	s/e	total no. of steps per statement
Algorithm Mul(a,b,c,n) { for i:=1 to n do { for j:=1 to n do { c[i,j]:=0; for k:=1 to n do { c[i,j]:= c[i,j]+a[i,k]* b[k,j]; } } } } } 	n+1 n(n+1) n² n²(n+1) n³	1 1 1 1 1	n+1 n²+ n n² n³+ n² n³
Total step count =			2n ³ +3n ² +2n+1

EXAMPLE-4:- Algorithm which takes input n and computes the nth Fibonacci number and prints it.

->The Fibonacci sequence is: 0, 1, 1,2,3,5 . . .

->If we name the first term of the sequence as f_1 and second term as f_2 , then $f_1=0$ and $f_2=1$, and in general,
 $f_n=f_{n-1}+f_{n-2}$, where $n\geq 3$.

Statement	No. of times of execution (frequency)		s/e	total no. of steps per statement	
	n<=2	n>2		n<=2	n>2
Algorithm Fibonacci(n)					
{					
if(n<=2) then	1	1	1	1	1
write(n-1);	1	0	1	1	0
else					
{					
fn-2:=0; fn-1:=1;	0	1	2	0	2
for i:=3 to n do	0	n-1	1	0	n-1
{					
fn:=fn-1+fn-2;	0	n-2	1	0	n-2
fn-2:=fn-1;	0	n-2	1	0	n-2
fn-1:=fn;	0	n-2	1	0	n-2
}					
write(fn);	0	1	1	0	1
}					
}					
Total step count =				2	4n-3

Order (or, Rate) of growth of running time:

→Our motivation to determine step count is to compare the running times of two alternative algorithms that perform the same task and also to predict how the running time of an algorithm grows as its input size changes.

So, would like to determine the order of growth of the running time of an algorithm (in terms of its input size), rather than determining its exact running time. For this purpose, we use asymptotic notations.

→The logic behind the above idea is as follows:

For large enough inputs, the multiplicative constants and lower-order terms of an exact running time are dominated by the higher-order term.

EXAMPLE:-

→Suppose the runtime of an algorithm is $6n^2+100n+300$.

- The term $100n+300$ becomes less significant to the total value of the function as n grows larger. So, we can drop the term $100n+300$. And we are left with only $6n^2$. We can also drop the coefficient 6. And we can say that the running time of this algorithm grows in proportion to n^2 .

Asymptotic notations:-

We will use asymptotic notations primarily to describe the running times of algorithms.

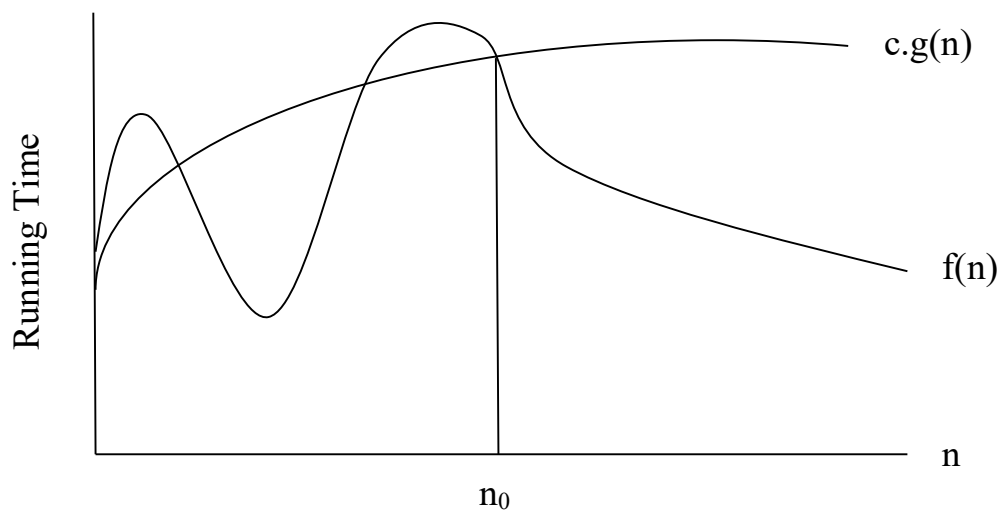
However, asymptotic notations actually apply to functions.

Asymptotic notation is used to describe the limiting behaviour of a function, when its argument tends towards a particular value (often infinity) usually in terms of simpler functions.

While analysing the runtime of an algorithm, we should not only determine how long the algorithm takes in terms of its inputs size but also should focus on how fast this runtime function grows with the input size which is facilitated by asymptotic notations.

(1) Big-Oh notation: - (O)

Let $f(n)$ and $g(n)$ be functions mapping non-negative integers to non-negative real numbers. We can say that $f(n)$ is $O(g(n))$ iff there exists a real constant $c > 0$ and an integer constant $n_0 > 0$ such that $f(n) \leq cg(n)$ for all $n \geq n_0$.



Ex:-

1) $3n+2=O(n)$

Here,

$$f(n)=3n+2$$

$$g(n)=n.$$

$3n+2=O(n)$ as $3n+2 \leq 4n$ for all $n \geq 2$, where $c=4$ and $n_0=2$.

2) $3n+3=O(n)$

3) $100n+6=O(n)$, as $100n+6 \leq 101n$, for all $n \geq 6$.

4) $10n^2+4n+2=O(n^2)$, as $10n^2+4n+2 \leq 11n^2$, for all $n \geq 5$. (or) $10n^2+4n+2 \leq 16n^2$, for all $n \geq 1$.

5) $6 \cdot 2^n + n^2 = O(2^n)$, as $6 \cdot 2^n + n^2 \leq 7 \cdot 2^n$, for all $n \geq 4$.

6) $3n+3=O(n^2)$, as $3n+3 \leq 3n^2$, for all $n \geq 1$ (**or**) $3n+3 \leq n^2$, for all $n \geq 4$

7) $2^{100}=O(1)$, as $2^{100} \leq 2^{100} \cdot 1$, for all $n \geq 1$.

Note:- We write $O(1)$ to mean a computing time of constant.

Theorem:- If $f(n)=a_m n^m + \dots + a_1 n + a_0$, then $f(n) = O(n^m)$.

Proof:-

$$\begin{aligned} f(n) &= \sum_{i=0}^m a_i n^i \\ &\leq \sum_{i=0}^m |a_i| n^i = n^m \sum_{i=0}^m |a_i| n^{i-m} \\ &\leq n^m \sum_{i=0}^m |a_i|, \text{ for all } n \geq 1. \\ &\leq c \cdot g(n), \text{ where } c = \sum_{i=0}^m |a_i| \text{ and } g(n) = n^m. \end{aligned}$$

So, $f(n) = O(n^m)$.

Ex:- $10n^2+4n+2=O(n^2)$ as $10n^2+4n+2 \leq 16n^2$ for all $n \geq 1$.

→ There are several functions $g(n)$ for which $f(n)=O(g(n))$ is true. The statement $f(n)=O(g(n))$ states that $g(n)$ is only an upper bound on the value of $f(n)$ for all $n \geq n_0$. For the statement $f(n)=O(g(n))$ to be meaningful, $g(n)$ should be as small function as possible (i.e., least upper bound) for which $f(n)=O(g(n))$ is true.

So, while we often say that, $3n+3=O(n)$ and $10n^2+4n+2=O(n^2)$, we almost never say that, $3n+3=O(n^2)$ or $10n^2+4n+2=O(n^3)$, even though both of these statements are true.

Frequently used Efficiency classes:

→ $O(1)$ is called Constant time.

→ $O(n)$ is called Linear time.

→ $O(n^2)$ is called Quadratic time.

→ $O(n^3)$ is called Cubic time.

→ $O(n^k)$, $k \geq 1$; is called Polynomial time.

→ $O(2^n)$ and $O(a^n)$ are called Exponential time.

→ $O(\log n)$ is called Logarithmic time.

→ For sufficiently large values of n , the following relationship holds among efficiency classes:

$Constant < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$

Ex:- $n^2+n \log n+n+4=O(n^2)$.

$3n+2 \neq O(1)$, as $3n+2$ is not less than or equal to any constant c for all $n \geq n_0$.

$10n^2+4n+2 \neq O(n)$.

Problem1: (GATE-2017 Set1)

Consider the following functions from positives integers to real numbers:

$10, \sqrt{n}, n, \log_2 n, 100/n$.

The CORRECT arrangement of the above functions in increasing order of asymptotic complexity is:

(A) $\log_2 n, 100/n, 10, \sqrt{n}, n$

(B) $100/n, 10, \log_2 n, \sqrt{n}, n$

(C) $10, 100/n, \sqrt{n}, \log_2 n, n$

(D) $100/n, \log_2 n, 10, \sqrt{n}, n$

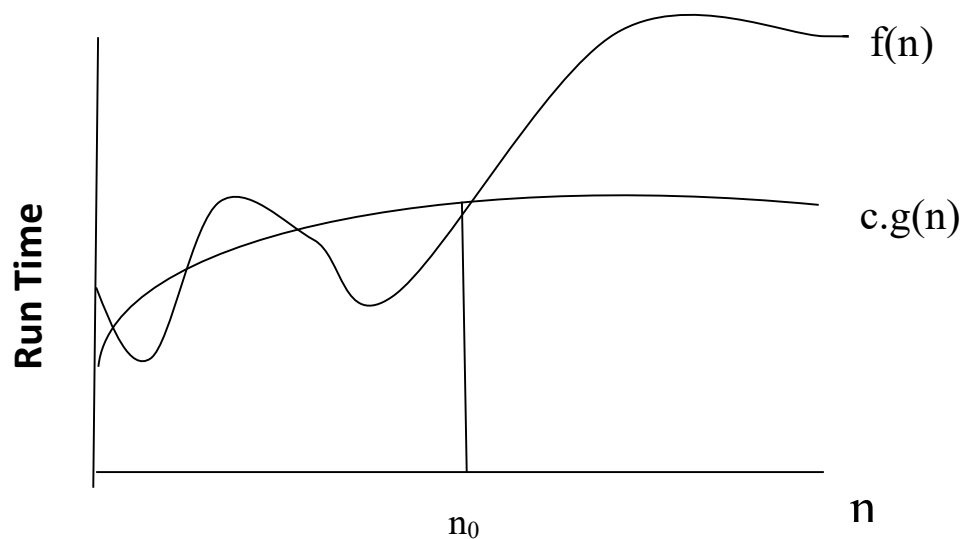
NOTE:-

1) $n! = O(n^n)$

2) $\log n^3 = O(\log n)$.

2) Big-omega notation(Ω):-

Let $f(n)$ and $g(n)$ be functions mapping non-negative integers to non-negative real numbers. We say that $f(n)$ is $\Omega(g(n))$ iff there exists a real constant $c > 0$ and an integer constant $n_0 > 0$ such that $f(n) \geq c \cdot g(n)$ for all $n \geq n_0$.

**Ex:-**

1) $3n+2 = \Omega(n)$ as, $3n+2 \geq 3n$ for all $n \geq 1$ where $c=3$ and $n_0=1$.

2) $3n+3 = \Omega(n)$ as, $3n+3 \geq 3n$ for all $n \geq 1$.

3) $100n+6 = \Omega(n)$ as, $100n+6 \geq 100n$ for all $n \geq 1$.

4) $10n^2+4n+2 = \Omega(n^2)$ as, $10n^2+4n+2 \geq n^2$ for all $n \geq 1$, where $c=1$ and $n_0=1$.

5) $6 \cdot 2^n + n^2 = \Omega(2^n)$ as, $6 \cdot 2^n + n^2 \geq 2^n$ for all $n \geq 1$.

6) $3n+3 = \Omega(1)$ since $3n+3 \geq 1$ for all $n \geq 1$

but $3n+3 \neq O(1)$.

7) $10n^2+4n+2 = \Omega(n^2) = \Omega(n) = \Omega(1)$.

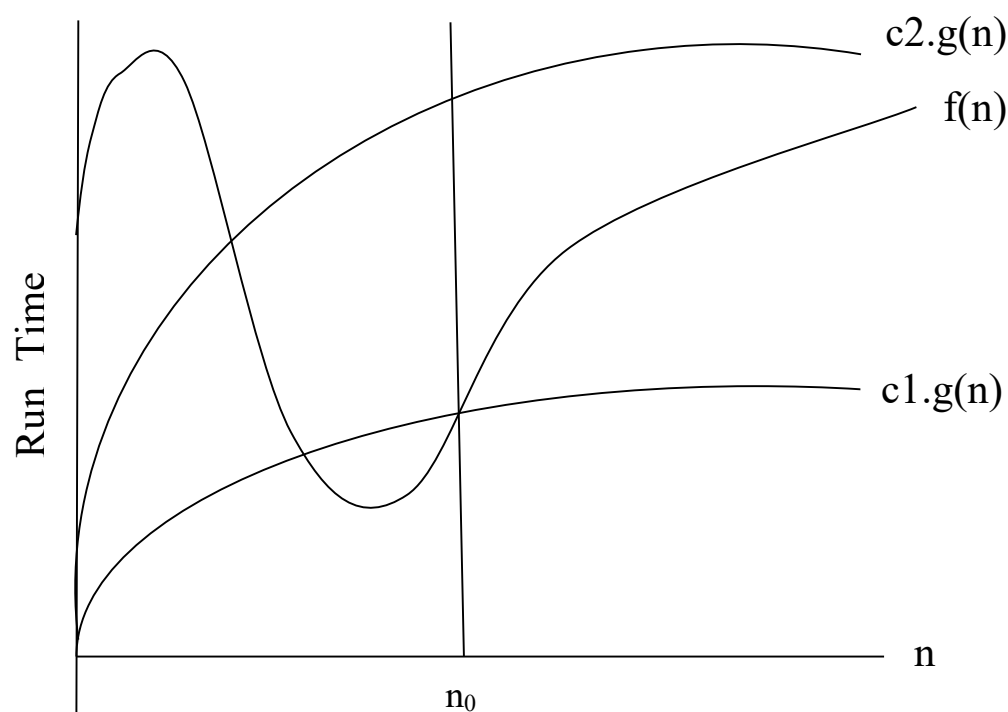
There are several functions $g(n)$ for which $f(n) = \Omega(g(n))$ is true. The function $g(n)$ is only a lower bound on $f(n)$. For the statement, $f(n) = \Omega(g(n))$ to be meaningful, $g(n)$ should be as large function as possible (i.e., largest lower bound) for which the statement, $f(n) = \Omega(g(n))$ is true.

→ So, while we say that, $3n+3 = \Omega(n)$ and $10n^2+4n+2 = \Omega(n^2)$, we almost never say that $3n+3 = \Omega(1)$ or $10n^2+4n+2 = \Omega(n)$ even though both of these statements are correct.

Big-Theta notation(Θ):-

Let $f(n)$ and $g(n)$ be functions mapping non-negative integers to non-negative real numbers. We say that $f(n)$ is $\Theta(g(n))$ iff there exist two real constants $c_1 > 0$ and $c_2 > 0$ and an integer constant $n_0 > 0$ such that $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$. That means, $f(n) = \Theta(g(n))$ iff $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

We can say that $f(n) = \Theta(g(n))$ iff $g(n)$ is both an upper bound and lower bound on $f(n)$.

**Ex:-**

1) $3n+2 = \Theta(n)$ as, $3n+2 \geq 3n$ and $3n+2 \leq 4n$ for all $n \geq 2$, where $g(n)=n$, $c_1=3$, $c_2=4$ and $n_0=2$.

2) $3n+3 = \Theta(n)$ since $3n+3 = O(n) = \Omega(n)$

3) $10n^2+4n+2 = \Theta(n^2)$.

4) $6 \cdot 2^n + n^2 = \Theta(2^n)$.

5) $10 \cdot \log n + 4 = \Theta(\log n)$.

6) $3n+2 \neq \Theta(1)$ since $3n+2 = \Omega(1)$ but $3n+2 \neq O(1)$

- 7) $3n+3 \neq \Theta(n^2)$.
 8) $10n^2+4n+2 \neq \Theta(n)$ and $10n^2+4n+2 \neq \Theta(1)$.
 9) $6 \cdot 2^n + n^2 \neq \Theta(n^2)$ and $6 \cdot 2^n + n^2 \neq \Theta(1)$.

Little-oh notation(o):- ($f(n) < g(n)$)

The function $f(n) = o(g(n))$ iff $\lim_{n \rightarrow \infty} f(n)/g(n) = 0$.

Ex:-

1) $3n+2 = o(n^2)$ since $\lim_{n \rightarrow \infty} (3n+2)/n^2 = 0$. [since $(3/n) + (2/n^2) = 0$ when $n = \infty$].

2) $3n+2 = o(n \log n)$ since $\lim_{n \rightarrow \infty} ((3n+2)/n \log n) = \lim_{n \rightarrow \infty} \left\{ \left(\frac{3}{\log n} \right) + \left(\frac{2}{n \log n} \right) \right\} = 0$.

3) $6 \cdot 2^n + n^2 = o(3^n)$ since $\lim_{n \rightarrow \infty} (6 \cdot 2^n + n^2)/(3^n) = \lim_{n \rightarrow \infty} \left\{ \frac{6 \cdot 2^n}{3^n} + n^2/3^n \right\} = 0$.

$= \lim_{n \rightarrow \infty} \left\{ 6 \cdot \left(\frac{2}{3} \right)^n + n^2/3^n \right\} = \lim_{n \rightarrow \infty} \{ n^2/3^n \}$

Using L'hospital's rule: (i.e., taking derivatives on both numerator and denominator)

$\lim_{n \rightarrow \infty} \{ n^2/3^n \} = \lim_{n \rightarrow \infty} \{ 2n/(\ln(3)3^n) \} = \lim_{n \rightarrow \infty} \{ (2/\ln(3)) \cdot 1/(\ln(3)3^n) \} = 0$

4) $3n+2 \neq o(n)$ since $\lim_{n \rightarrow \infty} (3n+2)/n = \lim_{n \rightarrow \infty} 3 + \left(\frac{2}{n} \right) = 3 \neq 0$.

Little omega notation(ω):- ($f(n) > g(n)$)

The function $f(n) = \omega(g(n))$ iff $\lim_{n \rightarrow \infty} g(n)/f(n) = 0$.

Ex:-

1) $3n+2 = \omega(1)$ since $\lim_{n \rightarrow \infty} 1/(3n+2) = 0$.

2) $10n^2+4n+2 = \omega(n)$ since $\lim_{n \rightarrow \infty} n/(10n^2+4n+2) = 0$.
 $= \omega(1)$
 $\neq \omega(n^2)$

Problem: - (GATE-2015 Set3 Question)

Consider the equality $\sum_{i=0}^n i^3 = X$ and the following choices for X

- I. $\Theta(n^4)$
- II. $\Theta(n^5)$
- III. $O(n^5)$
- IV. $\Omega(n^3)$

The equality above remains correct if X is replaced by

- (A) Only I
- (B) Only II
- (C) I or III or IV but not II
- (D) II or III or IV but not I

Hint:

$$\sum_{1 \leq i \leq n} i^k = \frac{n^{k+1}}{k+1} + \frac{n^k}{2} + \text{lower order terms}$$

Problem: - (GATE-2015 Set3 Question)

Let $f(n) = n$ and $g(n) = n^{(1+\sin n)}$, where n is a positive integer. Which of the following statements is/are correct?

- I. $f(n) = O(g(n))$
- II. $f(n) = \Omega(g(n))$

- (A) Only I
- (B) Only II
- (C) Both I and II
- (D) Neither I nor II

Answer: (D)

Explanation: The value of sine function varies from -1 to 1.

For $\sin = -1$ or any other negative value, I becomes false.

For $\sin = 1$ or any other positive value, II becomes false.

Asymptotic Notations and Time Complexities of Algorithms:

Problem-1:-

Give a big-O characterization in terms of n, of the running time of the following code.

```
Sum:=0           ----->1
for i:=1 to n do  ----->n+1
sum= sum+i;      ----->n
```

Running time= $2n+2 = O(n)$.

Problem-2:-

Give a big oh characterization in terms of n of the running time of the loop 1 method shown in the following algorithm.

Algorithm loop1(n)

```
{
p:=1           ----->1
for i:=1 to 2n do ----->2n+1
  p:=p*i       ----->2n
}
```

Problem-3:- (C code fragment)

Algorithm loop2(n)

```
{
for(i=n; i>=1;)
{
  i=i/2;
  print(i);
}
```

Time complexity = Number of times the loop executed.

Initial value of i is n and final value of i is 1.

If the loop is executed x times, then $n/2^x = 1 \Rightarrow n = 2^x \Rightarrow x = \log_2 n \Rightarrow x = O(\log n)$

Problem-4:- (C code fragment)

Algorithm loop3(n)

```
{
for(j=1; j<=n;)
{
  j=j*2;
  print(j);
}
```

Time complexity = Number of times the loop executed.

Initial value of j is 1 and final value of j is n.

If the loop is executed x times, then $2^x = n \Rightarrow x = \log_2 n \Rightarrow x = O(\log n)$

Problem-5:- (C code fragment)

```
i=1;
S=0;
while(S<=n)
{
  S=S+i;
  i++;
}
```

Time complexity = Number of times the loop executed.

Initial value of S is 0 and final value of S is n.

If the loop is executed k times, then final value of $S = 0+1+2+3+\dots+k = n$
 $k(k+1)/2 = n \Rightarrow k = O(\sqrt{n})$.

Practice Problem:

What is the complexity of the following code?

```
1. sum=0;
2.   for(i=1;i<=n;i*=2)
3.     for(j=1;j<=n;j++)
4.       sum++;
```

- A. $O(n^2)$
- B. $O(n \log n)$
- C. $O(n)$
- D. $O(n \log n \log n)$

Practice Problem:

Consider the following C function.

```
int fun(int n)
{
    int i, j;
    for (i = 1; i <= n ; i++)
    {
        for (j = 1; j < n; j += i)
        {
            printf("%d %d", i, j);
        }
    }
}
```

Time complexity of fun in terms of θ notation is:

- (A) $\theta(n \sqrt{n})$
- (B) $\theta(n^2)$
- (C) $\theta(n \log n)$
- (D) $\theta(n^2 \log n)$

Practice Problem (GATE 2015 Set-1)

Consider the following C function.

```
int fun1 (int n)
{
    int i, j, k, p, q = 0;
    for (i = 1; i < n; ++i)
    {
        p = 0;
        for (j = n; j > 1; j = j/2)
            ++p;
        for (k = 1; k < p; k = k*2)
            ++q;
    }
    return q;
}
```

Which one of the following most closely approximates the return value of the function fun1?

- (A) n^3
- (B) $n (\log n)^2$
- (C) $n \log n$
- (D) $n \log(\log n)$

Practice Problem (GATE 2013)

Consider the following function:

```
int unknown(int n) {
    int i, j, k = 0;
    for (i = n/2; i <= n; i++)
        for (j = 2; j <= n; j = j * 2)
            k = k + n/2;
    return k;
}
```

The return value of the function is

- A. $\Theta(n^2)$
 - B. $\Theta(n^2 \log n)$
 - C. $\Theta(n^3)$
 - D. $\Theta(n^3 \log n)$
-

Problem-6: Present an algorithm that searches an unsorted array $a[1 : n]$ for the element x . If x is present then return a position in the array; else return 0. And analyse its time complexity.

Sol:

Algorithm Search(a, n, x)

```
{
    for i:=1 to n do
        if( $a[i]=x$ ) then
            return i;
    return 0;
}
```

The above algorithm may terminate in one iteration (i.e., 3 steps) if x is present in the first position, or it may take two iterations (i.e., 5 steps) if x is present in the second position, and so on.

In other words, knowing ‘n’ alone is not enough to estimate the runtime of the algorithm.

How to overcome this difficulty in determining the step count uniquely?

Explanation: -

When the chosen parameters are not adequate to determine the step count (or, time complexity) uniquely, we define 3 kinds of step counts (or, time complexities) :

- i.e., **Best case**
- Worst case**
- Average case**

Best case step count (or, Best case Time complexity):-

It is the minimum number of steps taken by the algorithm for *any* input of size n.
(OR)
It is smallest running time of the algorithm for *any* input of size n.

Worst case step count (or, Worst case Time complexity):-

It is the maximum number of steps taken by the algorithm for *any* input of size n.
(OR)
It is longest running time of the algorithm for *any* input of size n.

Average case step count (or, Average case Time complexity):-

It is the average number of steps taken by the algorithm on all instances of input with size n.
(OR)
It is running time of the algorithm for a random instance of input of size n.

→For the above algorithm, the best-case time complexity happens when the element x is present in the first position and the worst-case time complexity happens when the element x is present in the last position or if it is not present.

Statement	Best case	Worst case
	(Assuming that the element x is present in the first position)	(Assuming that the element x is present in the last position)
	total no. of steps per statement	total no. of steps per statement
Algorithm Search(a,n,x) { for i:=1 to n do if(a[i]=x) then return i; return 0; }	1 1 1 0	n n 1 0
Total step count =	3	2n+1

The best-case step count = 3 =Constant
= O(1) = Ω(1) = Θ(1)

The worst-case step count = 2n+1
= O(n) = Ω(n)= Θ(n)

The average case step count = (3+5+7+...+2n+1)/n = (3+5+7+...+2n-1+2n+1)/n
= ((1+3+5+7+...+2n-1)+2n)/n
= (n²+2n)/n
= n+2
= O(n) = Ω(n) = Θ(n)

Average Time complexity ≤ Worst-case Time complexity.

Note:

1. The worst-case running time of an algorithm gives us an upper bound on the running time of the algorithm for any input. We use ‘O’ notation to denote the upper bound on the running time of an algorithm.

2. The best-case running time of an algorithm gives us a lower bound on the running time of the algorithm for any input. We use ‘ Ω ’ notation to denote the lower bound on the running time of an algorithm.

3. We use ‘ Θ ’ notation to denote the running time of an algorithm if both the upper and lower bounds on the running time of the algorithm are same.

4. We generally concentrate on upper bound because knowing lower bound of an algorithm is of no practical importance.

So, we can say that the time complexity of the linear search algorithm is $O(n)$ because it gives the upper bound on the run time (i.e., it indicates the maximum run time).

It is also true that the time complexity of the linear search algorithm is $\Omega(1)$ because it gives the lower bound on the run time (i.e., it indicates the minimum run time).

But generally, we don’t express the time complexity of an algorithm, alone in terms of the lower bound on its run time because it doesn’t give any information about the upper bound on the run time (i.e., maximum running time) of the algorithm which is of prime importance for us.

Problem 7: (GATE-2007)

Consider the following C code segment:

```
int IsPrime(n)
{
    int i, n;
    for(i=2; i<=sqrt(n); i++)
        if(n%i == 0)
            {printf("Not Prime\n"); return 0;}
    return 1;
}
```

Let $T(n)$ denotes the number of times the for loop is executed by the program on input n . Which of the following is TRUE?

- (A) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(\sqrt{n})$
- (B) $T(n) = O(\sqrt{n})$ and $T(n) = \Omega(1)$
- (C) $T(n) = O(n)$ and $T(n) = \Omega(\sqrt{n})$
- (D) None of the above

Problem 8: (GATE-2013)

Which one of the following is the tightest upper bound that represents the number of swaps required to sort n numbers using selection sort?

- (A) $O(\log n)$
- (B) $O(n)$
- (C) $O(n \log n)$
- (D) $O(n^2)$

Problem: Analyze the Time complexity of Insertion Sort

Algorithm InsertionSort (a, n)

```
{
    // sort the array a[1:n] into non decreasing order.
    for i := 2 to n do
    {
        key:=a[i];
        // Insert a[i] into the sorted part of the array, i.e., a [1: i-1]
        j:=i-1;
        while(j≥1 and a[j]>key) do //Searching for the position of key and inserting it in its place by shifting
                                //the larger elements right side
        {
            a[j+1]:=a[j]; // move a[j] to its next position in the right side
            j:=j-1;
        }
        a[j+1]:=key;
    }
}
```

In Worst-case (when the elements are in descending order), the time complexity is $O(n^2)$.
 In Best-case (when the elements are in ascending order), the time complexity is $O(n)$.
 In Average-case, the time complexity is $O(n^2)$.

Disjoint Sets

Suppose we have some finite universe of n elements, U , out of which sets will be constructed. These sets may be empty or contain any subset of the elements of U . We shall assume that the elements of the sets are the numbers 1, 2, 3, ..., n .

We assume that the sets being represented are pair wise disjoint, i.e., if S_i and S_j ($i \neq j$) are two sets, then there is no element that is in both S_i and S_j .

For example, when $n = 10$, the elements can be partitioned into three disjoint sets, $S_1 = \{1, 7, 8, 9\}$, $S_2 = \{2, 5, 10\}$, and $S_3 = \{3, 4, 6\}$.

→ The following two operations are performed on the disjoint sets:

1) Union:

If S_i and S_j are two disjoint sets, then their union $S_i \cup S_j = \{\text{all elements } x \text{ such that } x \text{ is in } S_i \text{ or } S_j\}$. Thus, in our example, $S_1 \cup S_2 = \{1, 7, 8, 9, 2, 5, 10\}$.

Since we have assumed that all sets are disjoint, we can assume that following the union of S_i and S_j , the sets S_i and S_j do not exist independently; that is, they are replaced by $S_i \cup S_j$ in the collection of sets.

2) Find(i): Given the element i , find the set containing i .

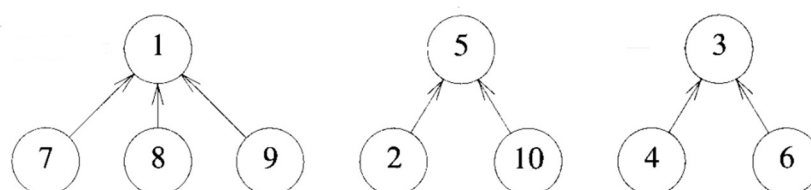
Thus, in our example, 4 is in set S_3 , and 9 is in set S_1 .

So, $\text{Find}(4) = S_3$

$\text{Find}(9) = S_1$

To carry out these two operations efficiently, we represent each set by a tree.

One possible representation for the sets $S_1 = \{1, 7, 8, 9\}$, $S_2 = \{2, 5, 10\}$, and $S_3 = \{3, 4, 6\}$ using trees is given below:



Note: For each set, we have linked the nodes from the children to the parent.

In presenting the UNION and FIND algorithms, we ignore the set names and identify sets just by the roots of the trees representing sets. This simplifies the discussion.

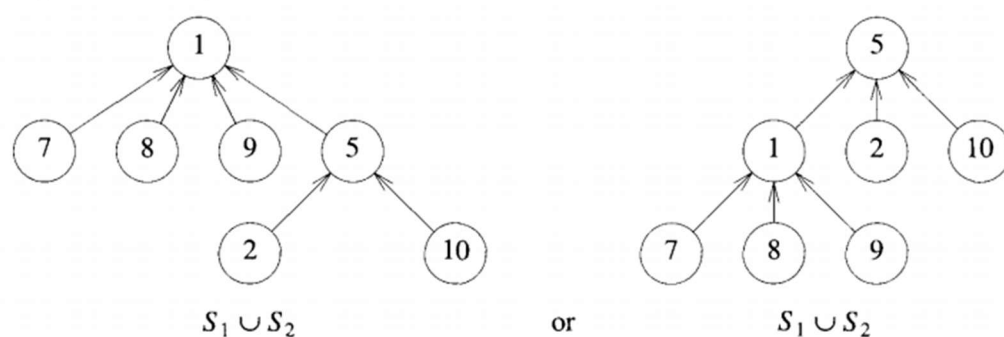
The operation of $\text{Find}(i)$ now becomes:

Determine the root of the tree containing element 'i'.

Ex: $\text{Find}(1)=1$, $\text{Find}(7)=1$, $\text{Find}(5)=5$, $\text{Find}(2)=5$, $\text{Find}(3)=3$, $\text{find}(6)=3$, and so on.

To obtain the union of two sets, all that has to be done is to link one of the roots to the other root. The function $\text{Union}(i, j)$ requires two trees with roots i and j to be joined.

The possible representations of $S_1 \cup S_2$:



Representing the tree nodes of all disjoint sets using a single array:

Since the universal set elements are numbered 1 through n , we represent the tree nodes of all sets using an array $P[1:n]$, where P stands for parent.